DOCUMENTATION TECHNIQUE

Documentation Technique & Architecture du Pipeline

Table des Matières

CLI To-Do List Manager - Technical Specification & Architecture Document

1. Executive Summary & Architecture Overview

1.1 Executive Brief

The CLI To-Do List Manager is a Python-based command-line application designed for efficient task management with persistent JSON-based storage. The system follows a layered architectural pattern consisting of a CLI dispatch layer for user interaction, a service-level logic layer for business rules, and a storage helper layer for data persistence. Core capabilities include task creation with sequential ID tracking, status updates, and dual-format (Human/JSON) listing.

1.2 Maturity Assessment

The project is logically structured with a high degree of foundational completeness. The implementation follows a strict phased approach (Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish). However, the absence of formalized acceptance criteria and explicit security/performance constraints for the JSON storage file necessitates a final validation pass.

Status: READY.

1.3 Technical Stack

- **Languages & Frameworks**:
- Python
- pyproject.toml (Tooling & Metadata)
- **Storage**:
- JSON (Flat-file persistence)

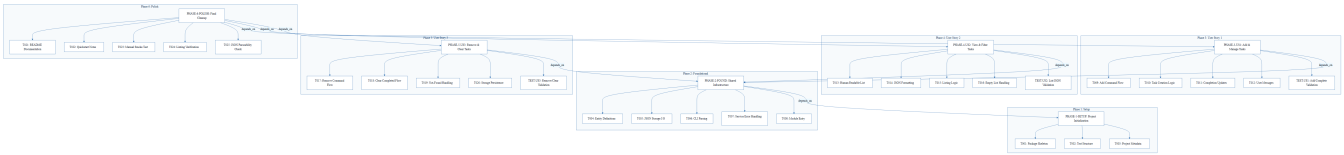
1.4 Architectural Constraints

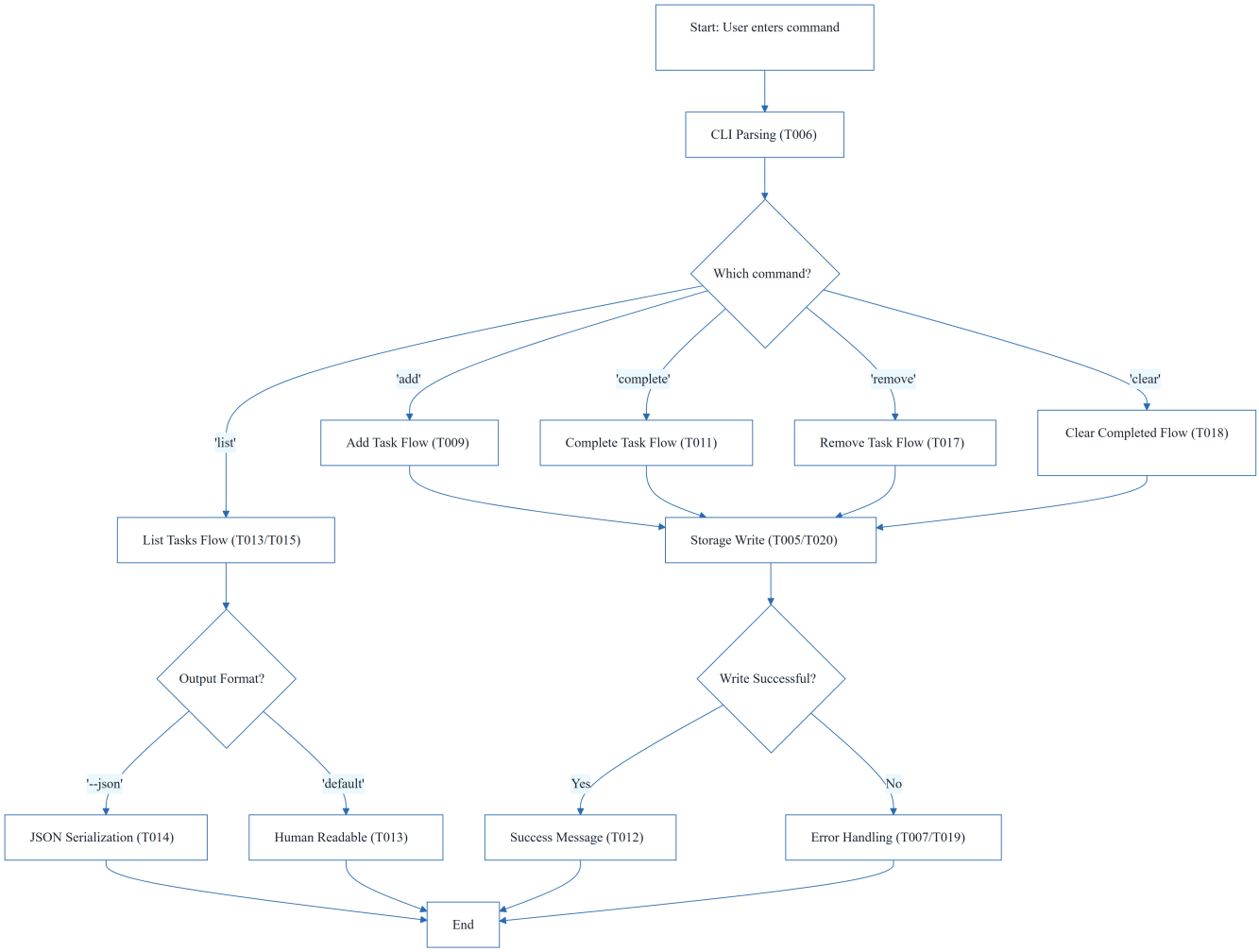
- **Sequential ID Assignment**: Task entities must use sequential integers for identification.
- **Strict JSON Serialization**: Data persistence must adhere to strict JSON formatting to ensure consistency.
- **Layered Dependency**: Mandatory completion of the Foundational layer before any User Story implementation.
- **Output Consistency**: The `--json` flag must always produce parseable JSON output.
- **Storage Location**: Path resolution must target `~/.todos.json`.

1.5 Critical Dependencies

- **Persistence**: Reliance on the `~/.todos.json` file for all data state.
- **ID Integrity**: Sequential ID dependency for precise task identification.
- **Execution Flow**: Strict hierarchy: Setup $\rightarrow$ Foundational $\rightarrow$ User Stories $\rightarrow$ Polish.
- **Serialization Bridge**: Integration between `src/todo_manager/models.py` serialization and CLI output formatting.
- **Referential Integrity**: Maintenance of task IDs during `remove` and `complete` operations.

2. Architecture Workflows

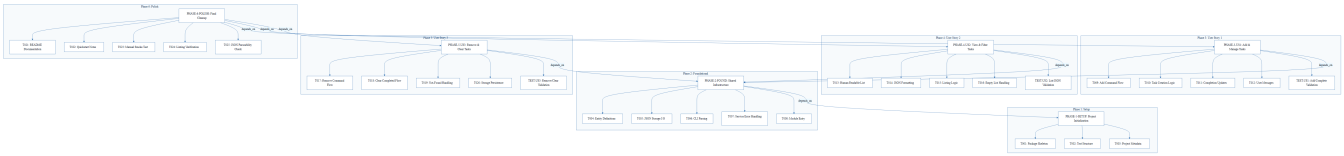




& Visual Diagrams

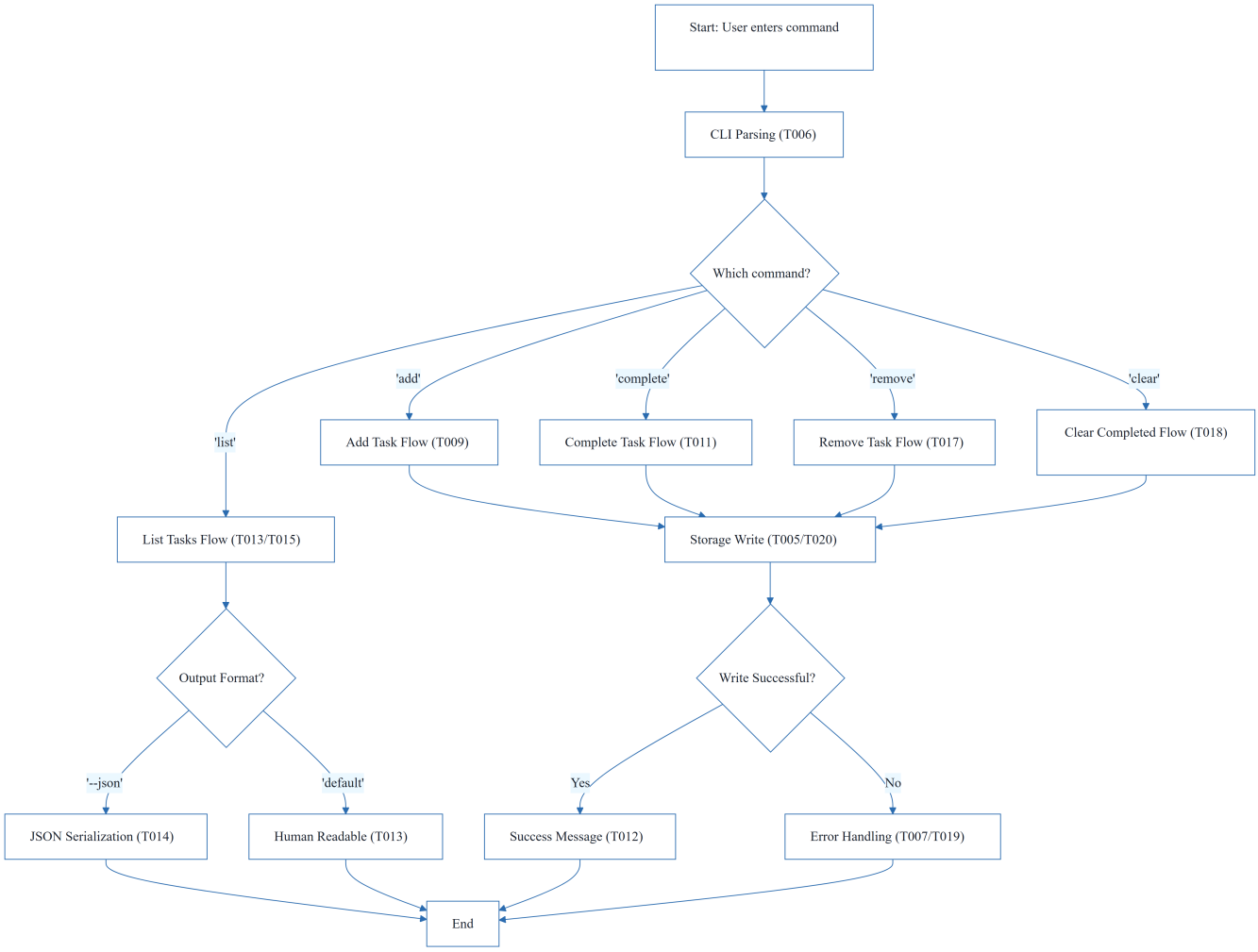
2.1 Project Implementation Traceability Map

This flowchart maps the dependency flow from initial setup through foundational layers to specific user story implementation and final polish.



2.2 CLI Command Execution Workflow

This flowchart models the general business logic for processing a CLI command, including the decision path for different user story actions.



3. Detailed Technical Specifications & Business Rules

3.1 Requirements Traceability

The following table provides an exhaustive mapping of all atomic identifiers to their implementation content.

ID	Description	Source Section	Status
T001	Create the Python package skeleton in src/todo_manager/**init**.py, **main**.py, cli.py, models.py, storage.py, and service.py	Phase 1: Setup	Completed
T002	Create the test directory structure in tests/unit/ and tests/integration/	Phase 1: Setup	Completed
T003	Add project metadata and tooling entry points in pyproject.toml	Phase 1: Setup	Completed
T004	Implement task entity definitions and serialization helpers in src/todo_manager/models.py	Phase 2: Foundational	Completed
T005	Implement JSON storage path resolution and file I/O helpers in src/todo_manager/storage.py	Phase 2: Foundational	Completed
T006	Implement shared command-line parsing and top-level dispatch in src/todo_manager/cli.py	Phase 2: Foundational	Completed
T007	Implement shared service-layer error handling and task collection utilities in src/todo_manager/service.py	Phase 2: Foundational	Completed
T008	Define module entry behavior for python -m todo*manager in src/todo*manager/**main**.py	Phase 2: Foundational	Completed
T009	Implement the add command flow in src/todo*manager/cli.py and src/todo*manager/service.py	Implementation for US1	Completed
T010	Implement task creation, sequential ID assignment, and created*at population in src/todo*manager/models.py	Implementation for US1	Completed
T011	Implement completion updates for existing tasks in src/todo_manager/service.py	Implementation for US1	Completed

ID	Description	Source Section	Status
T012	Add user-facing success and error messages for add and complete operations in src/todo_manager/cli.py	Implementation for US1	Completed
T013	Implement human-readable task listing output in src/todo_manager/cli.py	Implementation for US2	Completed
T014	Implement --json output formatting in src/todo_manager/cli.py using serialization helpers	Implementation for US2	Completed
T015	Add listing logic that preserves task order and status fields in src/todo_manager/service.py	Implementation for US2	Completed
T016	Handle the empty-list case with a clear message in src/todo_manager/cli.py	Implementation for US2	Completed
T017	Implement the remove command flow in src/todo*manager/cli.py and src/todo*manager/service.py	Implementation for US3	Pending
T018	Implement the clear command flow for removing completed tasks in src/todo_manager/service.py	Implementation for US3	Completed
T019	Add not-found handling for invalid task IDs in src/todo_manager/cli.py	Implementation for US3	Completed
T020	Ensure storage writes persist removals and clears safely in src/todo_manager/storage.py	Implementation for US3	Completed
T021	Document the CLI usage and storage behavior in README.md	Phase 6: Polish	Pending
T022	Add quickstart verification notes and examples in specs/001-cli-todo-manager/quickstart.md	Phase 6: Polish	Pending
T023	Run a manual smoke test of add, list, complete, remove, and clear against ~/.todos.json	Phase 6: Polish	Pending

ID	Description	Source Section	Status
T024	Verify linting and formatting expectations for the source files in src/todo_manager/	Phase 6: Polish	Pending
T025	Confirm the generated JSON output remains parseable for the todo list --json path	Phase 6: Polish	Pending
TEST-US1	Run `todo add "Task description"` and `todo complete` , confirm stored task list reflects the new task and status.	Phase 3: US1	N/A
TEST-US2	Run `todo list` and `todo list --json` and confirm output is readable or parseable JSON.	Phase 4: US2	N/A
TEST-US3	Run `todo remove` and `todo clear`, confirm targeting works while others remain intact.	Phase 5: US3	N/A

3.2 Security Rules

- **Input Sanitization**: While not explicitly detailed in the source, the system must handle invalid task IDs (T019) to prevent application crashes.
- **File Access**: The application is restricted to reading and writing the `~/.todos.json` file.

3.3 Data Models

- **Task Entity**: Defined in `src/todo_manager/models.py`.
- **Attributes**:
- `id`: Sequential integer.
- `description`: String.
- `status`: Boolean/Enum (Completed/Pending).
- `created_at`: Timestamp.
- **Persistence**: JSON array of task objects.

4. Project Governance & Structural Gaps

4.1 Structural Gaps

Gap	Priority	Remediation Advice
Acceptance Criteria	MEDIUM	While User Stories have goals and tests, a formal list of acceptance criteria for each story would improve validation.
Security & Performance Constraints	LOW	Document constraints on file size for `~/.todos.json` or input sanitization.
Open Questions & Uncertainties	LOW	No technical uncertainties were flagged in the task list.

4.2 Remediation & Workflow

The project follows an incremental delivery strategy:

1. **MVP First**: Complete Setup $\rightarrow$ Foundational $\rightarrow$ US1 $\rightarrow$ Validate.
2. **Incremental Delivery**: Sequentially add US2 and US3, validating each independently.
3. **Finalization**: Execute the Polish phase (T021-T025) to ensure documentation and code quality.

5. Technical & Domain Glossary (Terminology Reference)

Term	Category	Context Anchor	Project Definition
Checkpoint	TECHNICAL_STACK	PHASE-2-FOUND	A synchronization marker indicating that a mandatory structural layer is verified and subsequent development streams can be executed concurrently.
Foundational	TECHNICAL_STACK	PHASE-2-FOUND	The base architectural layer comprising shared I/O helpers and core entity logic that must be fully implemented before any feature-specific work.
Goal	BUSINESS_DOMAIN	PHASE-3-US1	The primary operational objective that defines a successful outcome for a specific user-centric feature set.
ID	BUSINESS_DOMAIN	T010	A unique sequential integer assigned to each entry to allow precise targeting for updates or deletions.

Term	Category	Context Anchor	Project Definition
JSON	TECHNICAL_STACK	T005	The lightweight data-interchange format used for persistent storage in the home directory and for machine-readable output.
MVP	BUSINESS_DOMAIN	PHASE-3-US1	The smallest viable set of functional capabilities, specifically restricted to creation and completion of entries.
Organization	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The logical grouping of implementation steps based on user-centric requirements to ensure modular testability.
Prerequisites	TECHNICAL_STACK	Tasks: CLI To-Do List Manager	The set of external design documents and contracts required before execution of the task list.
README	TECHNICAL_STACK	T021	The primary documentation file describing system usage and persistence behavior.
Setup	TECHNICAL_STACK	PHASE-1-SETUP	The initial project bootstrap phase including skeleton creation and tooling configuration.
T016	TECHNICAL_STACK	T016	The specific implementation step responsible for providing a non-empty response when the stored collection is void.
Tests	TECHNICAL_STACK	T002	The verification suite split into unit and integration directories to ensure logic correctness.
population in	TECHNICAL_STACK	T010	The act of assigning a timestamp value to the creation field during entity instantiation.
todo clear	BUSINESS_DOMAIN	T018	The specific command operation that purges all entries currently marked as finished from the persistent store.

Term	Category	Context Anchor	Project Definition
todo list	BUSINESS_DOMAIN	T013	The command operation used to retrieve and display the entire collection of entries in either human or machine-readable formats.
■■ CRITICAL	TECHNICAL_STACK	PHASE-2-FOUND	A high-priority blocking constraint indicating that no subsequent phase may be started until the current layer is fully verified.